Operációs rendszerek

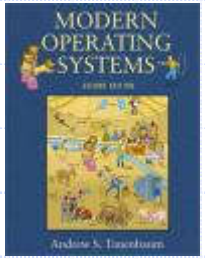
1. Bevezetés

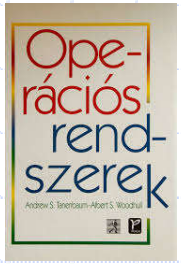
Felhasznált irodalom:

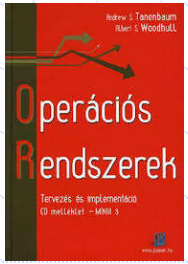
- **Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben**
- **Tanenbaum: Modern Operating Systems 2nd. Ed.**
- **Silberschatz, Galvin, Gagne: Operating System Concepts**

v4.0

Irodalom







- Kóczi-Kondorosi (szerk): Operációs rendszerek mérnöki megközelítésben. Panem Könyvkiadó, 2000. ISBN:9635452500
- Tanenbaum, Woodhul: Operációs rendszerek. Műszaki Könyvkiadó, 1999.
- Tanenbaum, Woodhull: Operációs rendszerek - tervezés és implementáció. Panem Kft., 2007. ISBN: 9789635454761
- Silberschatz, Galvin, Gagne: Operating System Concepts. Wiley, 2013. ISBN:9781118063330
- Tanenbaum, Bos: Modern Operating Systems. Pearson, 2015. ISBN: 9780133591620

Amit mindig szerettél volna tudni …

… de sosem merted megkérdezni.

C:\>

- Mi ez?
- Mi a „C:” belőle?
- Honnan származik?

Miért kell operációs rendszereket tanulni?

- Fogok én valaha operációs rendszert készíteni?
  - nem (valószínű)
- Szeretnék én valaha *hatékony* programot készíteni?
  - (remélhetőleg) igen
- Történelem
  - mi az az MS-DOS

Ebben segíthet ez a tárgy!

Miért kell operációs rendszereket tanulni?

- Ahol számítógép – ott operációs rendszer
- Hogyan működik a számítógép
  - Számítógép és operációs rendszer együtt
- Nagy rendszerek
  - Alrendszerek, modulok, rétegek
- Komplexitás kezelés absztrakcióval
  - CPU, memória, fájlok, szemaforok, erőforrások, interfészek
- Rendszertervezés
  - Teljesítmény vs. egyszerűség
  - HW vs. SW

Miért kell operációs rendszereket tanulni?

- Több program
  - Konkurens feldolgozás
  - Együttműködés
  - Versenyhelyzet
- Erőforrás kezelés
- Memória kezelés
- Háttértár kezelés
- Fájlrendszer használat
- Teljesítmény analízis

1. Az operációs rendszerekről általában

- Az operációs rendszerek célja
- Az operációs rendszerek fogalma
- Az operációs rendszerek feladatai

1.1 Az operációs rendszerek célja

- felhasználók kényelme
  - egyszerű, kényelmes, biztonságos használat (elsősorban kis gépeknél elsődleges, fontossága növekszik)
- hatékony gépkihasználás
  - adott idő alatt minél több program végrehajtása (nagy gépeknél elsődleges, fontossága csökken)

1.2 Az operációs rendszer fogalma

Definíció:

A számítógépen állandóan futó program

- Az a program, amely közvetlenül vezérli a gép működését (operációs rendszer magja, *kernel*).
- *Minden egyéb* alkalmazói program.

Definíció:

Az összes program, ami a szállítótól "operációs rendszer”-ként érkezik.

- A gyakorlati feladatokat ellátó programok.
- Minden ami a gép "általános" felhasználásához szükséges.
- Pl. grafikus felület, editor, számológép,...

1.3 Az operációs rendszerek feladatai

1. Végrehajtási környezet
2. Erőforrás kiosztás
3. Vezérlő program

Operációs rendszer feladatai 1

Végrehajtási környezet definíció:

Olyan környezet, ahol a felhasználók és programjaik hasznos munkát végezhetnek.

- A számítógép hardver szolgáltatásainak bővítése
- Elrejti a „piszkos” részleteket, könnyű felhasználhatóságot biztosít

Operációs rendszer feladatai 2

Erőforrás kiosztás

- Kezeli a rendszer erőforrásait
  - CPU,
  - központi tár,
  - merevlemez, stb.
- Tulajdonságai:
  - hatékony
  - biztonságos
  - igazságos felhasználás.

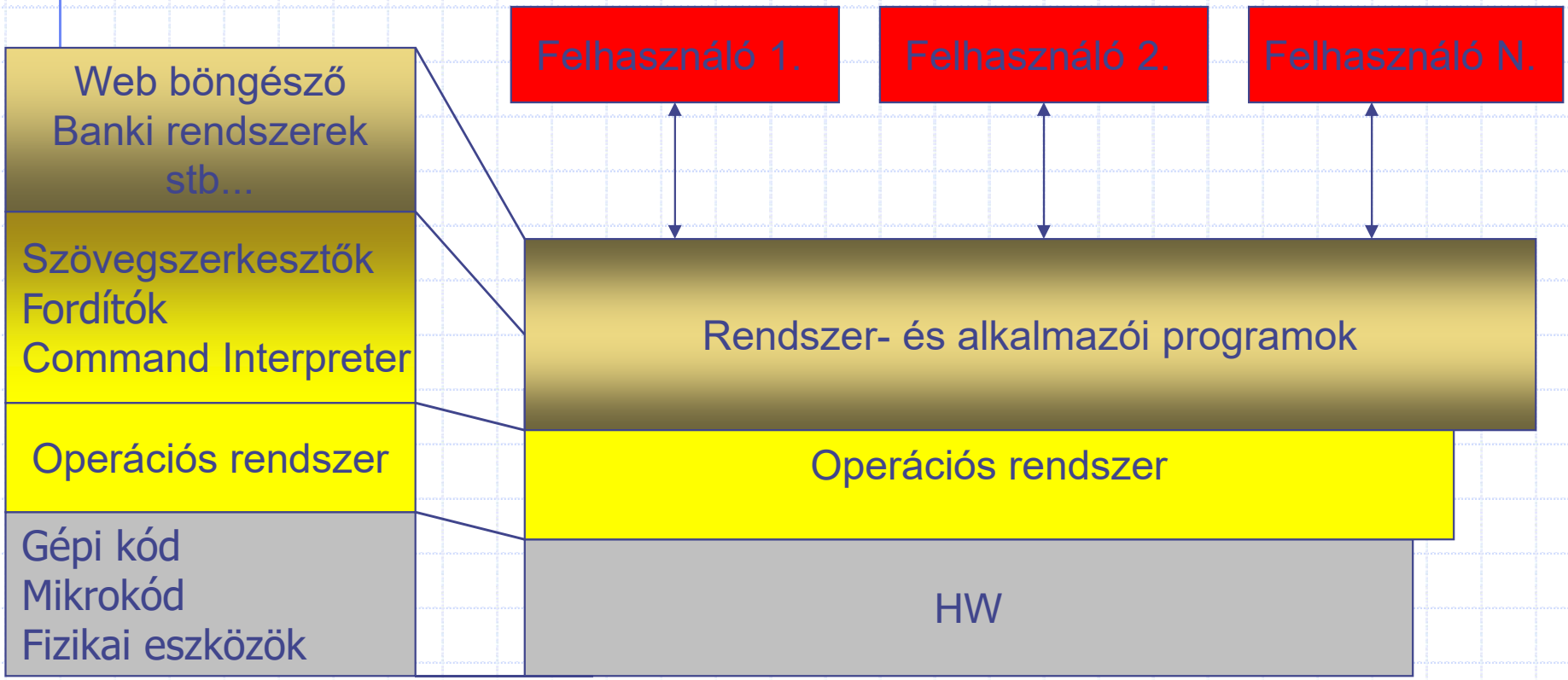
Operációs rendszer feladatai 3

Vezérlő program

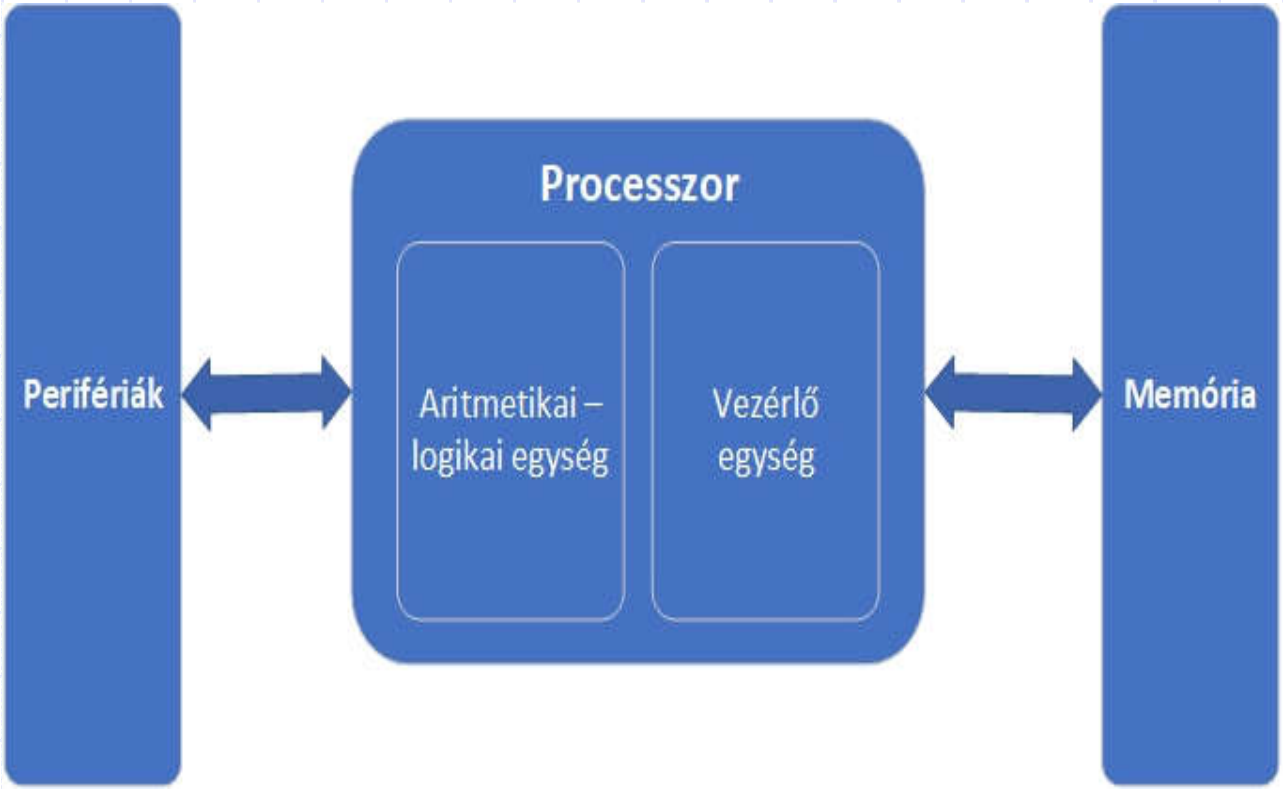
- vezérli a felhasználói programok működését,
- a felhasználói programok helyett vezérli a perifériák működését.

Számítógépes rendszerek

OS ma már alap!



Neumann architektúra



Statisztika

2023:

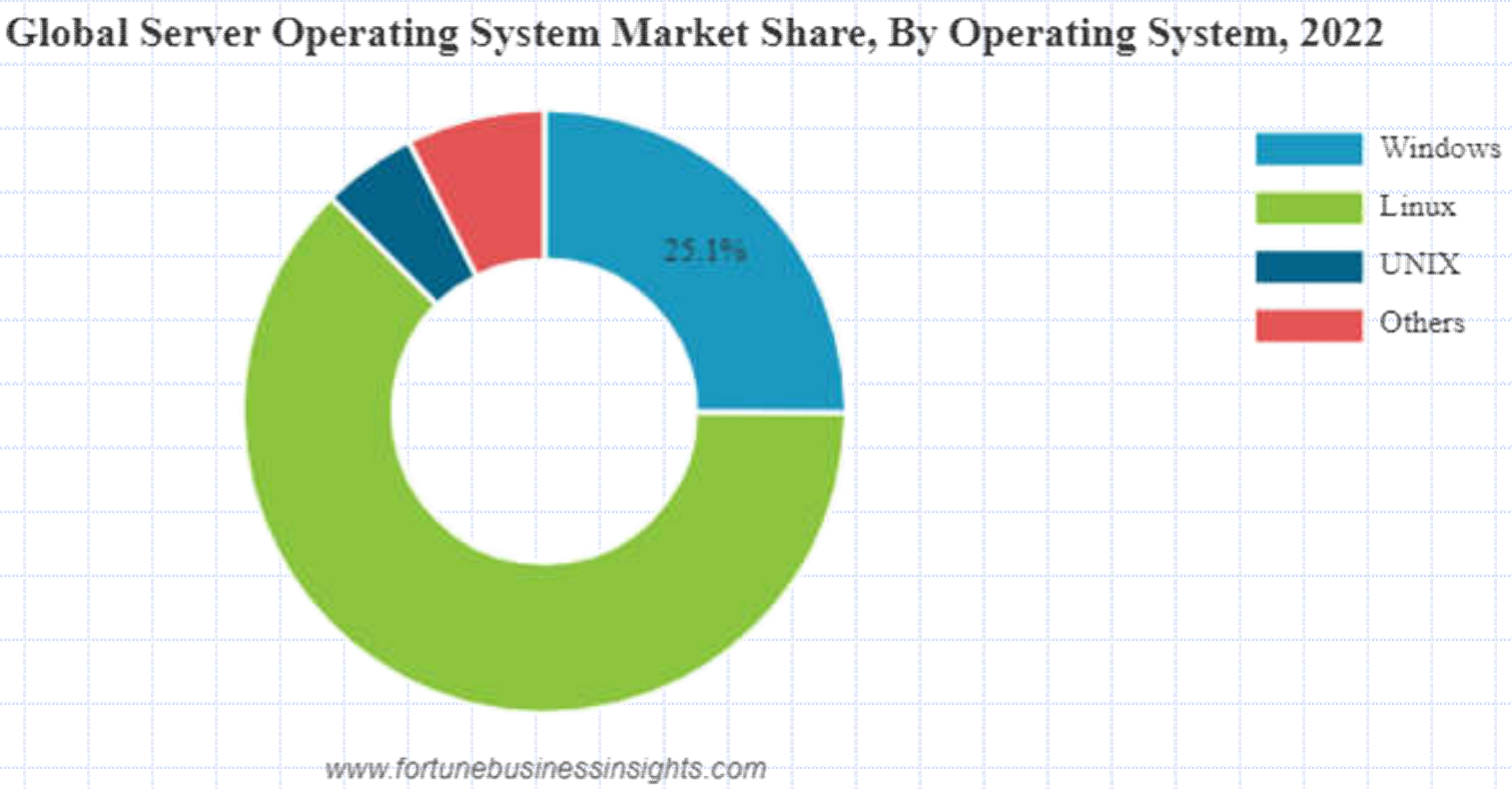
- Asztali operációs rendszerek (statista.com)

84,1%	20,42%	3,24%	3,12%

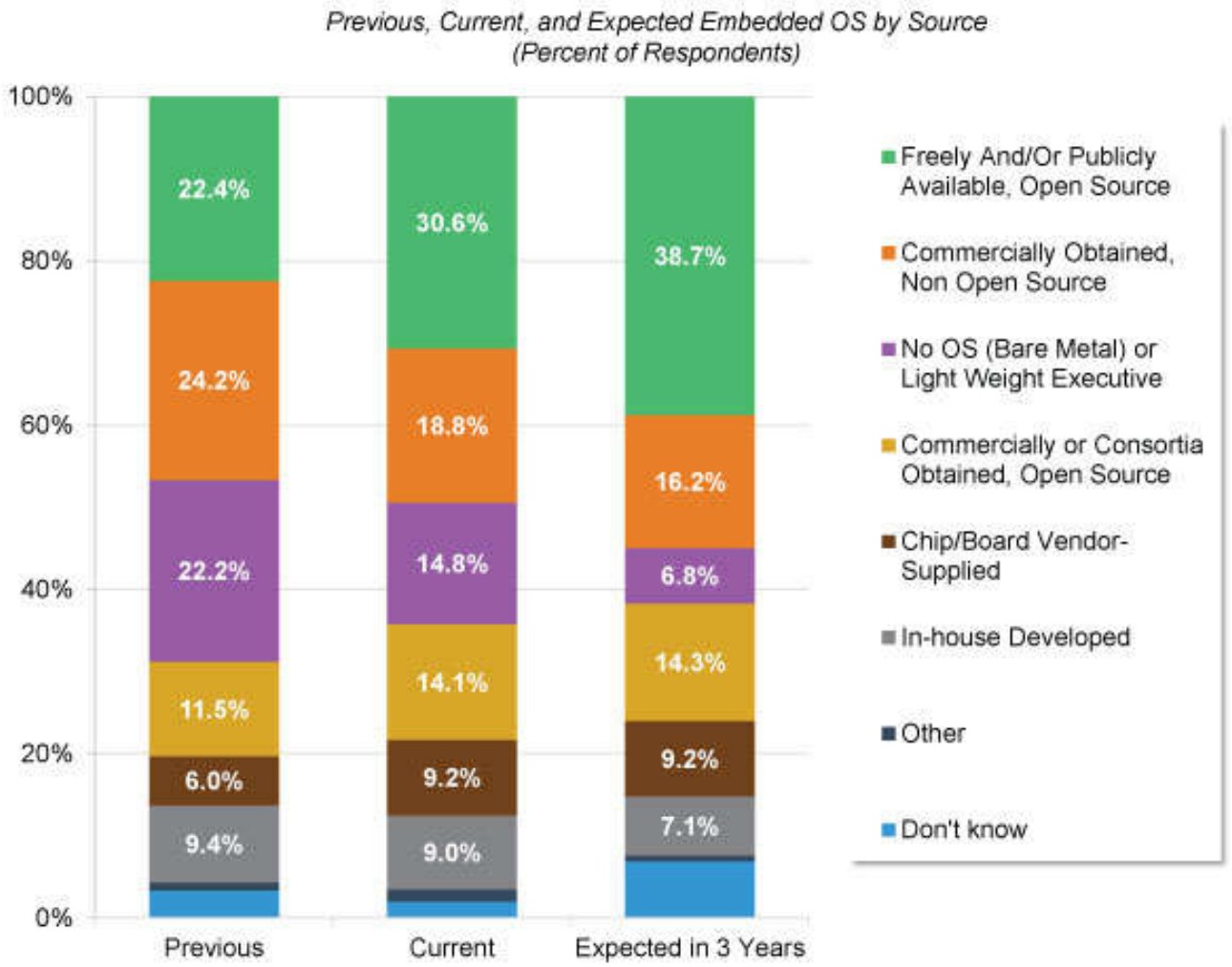
- Mobil operációs rendszerek (statcounter.com)

69,94%	29,32%	0,74%

Statisztika



Statisztika



2. Az operációs rendszerek története

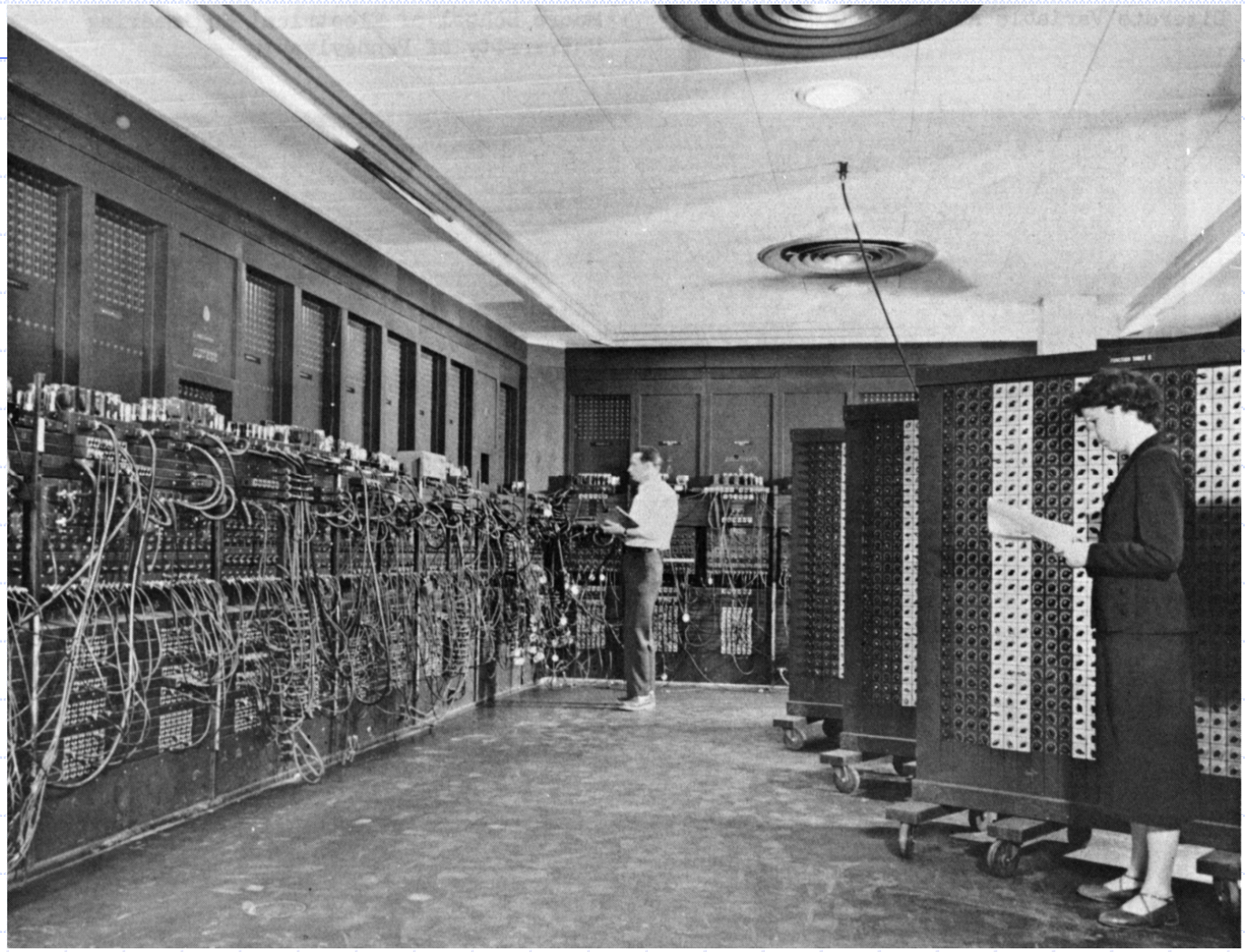
- Első generáció (1945-1955)
- Második generáció (1955-1965)
- Harmadik generáció (1965-1980)
- Negyedik generáció (1980-)

2.1 Első generáció

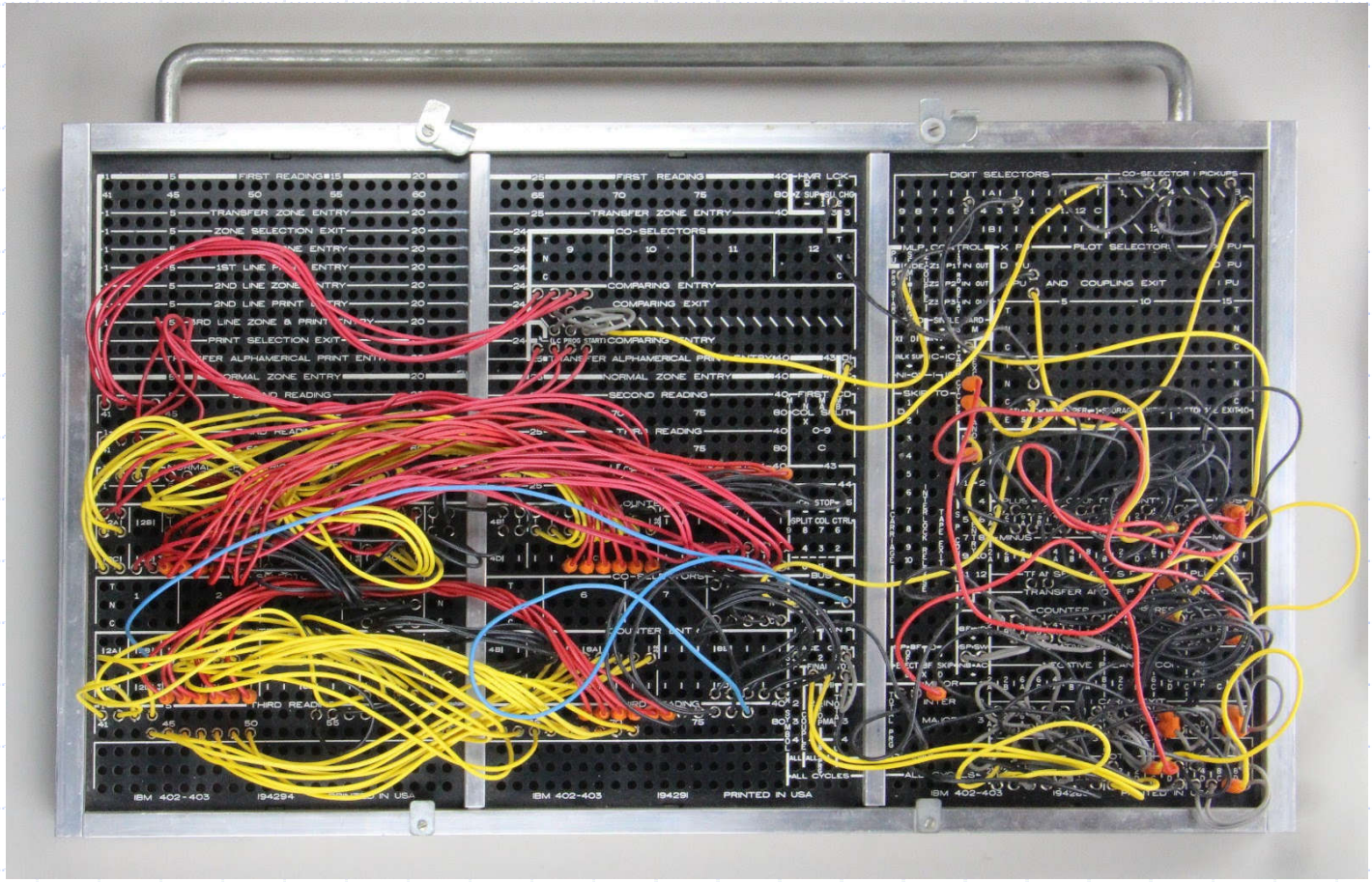
Nincs operációs rendszer (1945-1955)

- Hardver: nagy, drága, megbízhatatlan, plugboard vezérelt CPU, egyszerű perifériák
- Használat: programozó = operátor, kézi vezérlés, gépidő foglalás
- Mindent mindenki maga csinál, sok hiba, hibakeresés, ebből adódóan rossz kihasználtság.

ENIAC (Electronic Numerical Integrator And Computer)



Plugboard


FIRST READING
TRANSFER ZONE ENTRY
ZONE SELECTION EXIT
CO-SELECTORS
COMPARING ENTRY
COMPARING EXIT
NORMAL ZONE ENTRY
SECOND READING
THIRD READING
DIGIT SELECTORS
PILOT SELECTOR
AND COUPLING EXIT
COUNTER
IBM 402-403
PRINTED IN USA

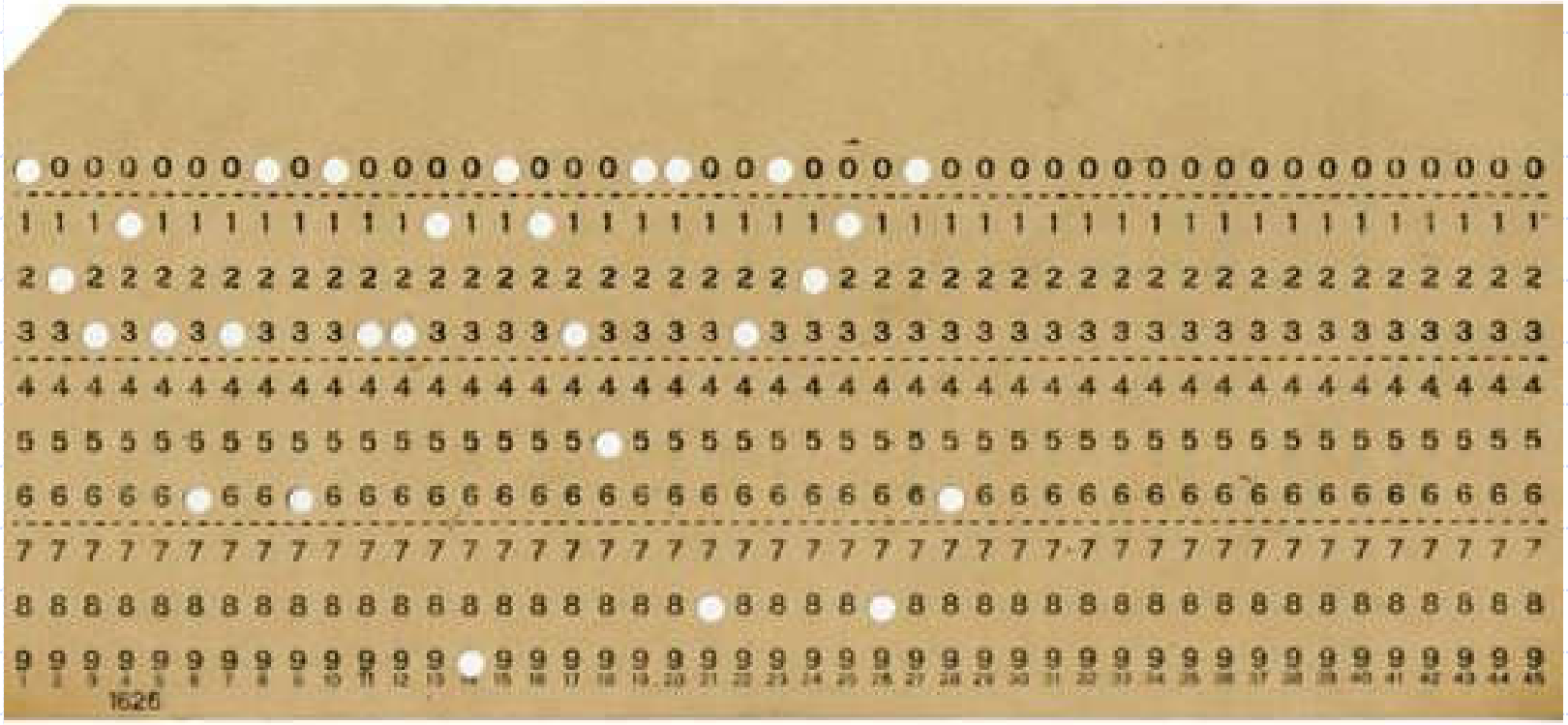
Programok

Egyszerű matematikai és számtani problémák

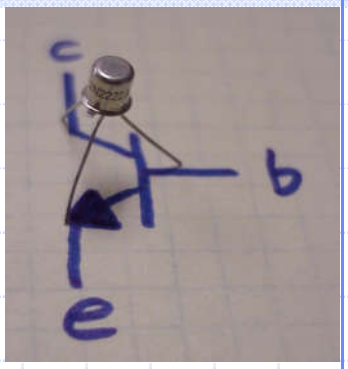
- Szinusz, koszinusz, logaritmus táblázatok
- Trajektóriák
- (Adószámítások)

Lyukkártya

Plugboard leváltása



2.2 Második generáció



Nincs operációs rendszer (1955-1965)

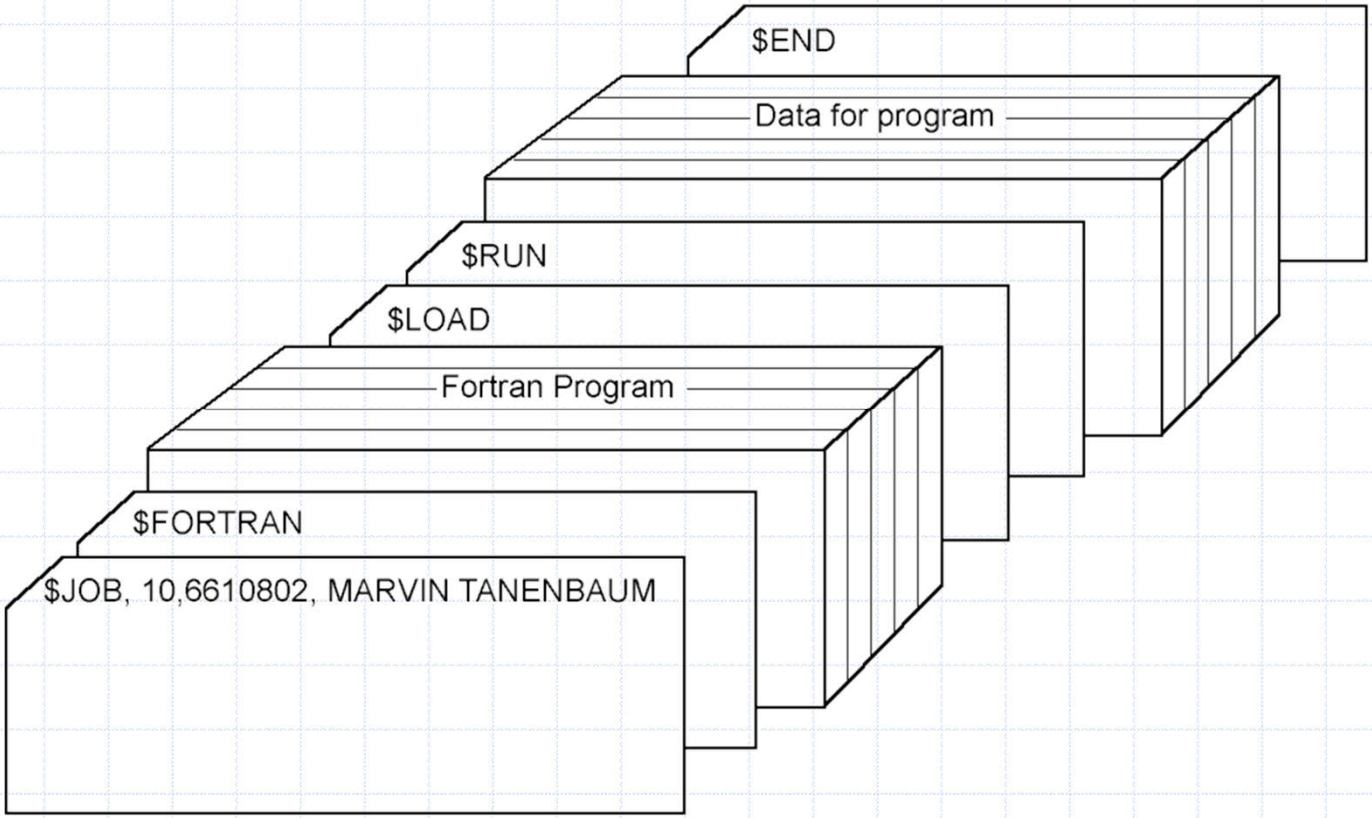
- Hardver: kisebb, olcsóbb, megbízhatóbb, egyszerű perifériák, mágnesszalag
- Használat: programozó és operátor szétválik
- Képzett operátor: gyorsabb gépkezelés, de hibakeresés még nincs
- Programozó: papír, lyukkártya, job, papír

Job

Program vagy programok halmaza

- Program
- Azonosítók
- Végrehajtási utasítások
- Fordító

Példa


$END
Data for program
$RUN
$LOAD
Fortran Program
$FORTRAN
$JOB, 10,6610802, MARVIN TANENBAUM

Job Control Language

$JOB - munka kezdete

$END - munka vége

$FORTRAN - FORTRAN fordító

$ASM - az assembler hívása

$LOAD - program betöltése

$RUN - program futtatása

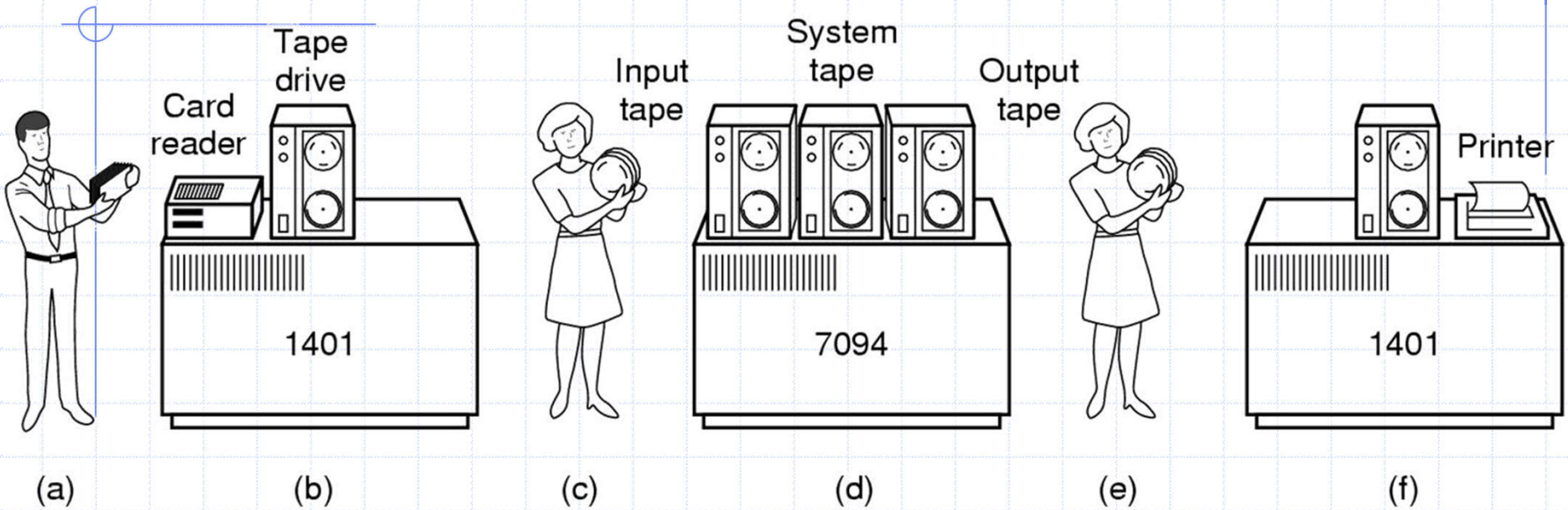
Kötegelt (batch) feldolgozás

- Probléma
  - Sok idő megy el a job összeállításával
  - Közben áll a drága CPU
- Ötlet
  - Off-line I/O műveletek
  - Az operátor kötegekbe (batch) csoportosítja a munkákat (jobs)
  - Egyszerre több job-ot visz be a gépbe
  - Gép sorosan feldolgozza a job-okat

Mágnesszalag megjelenése

- Bemenet szalagról (lényegesen gyorsabb mint a kártya). Egy másik gép végzi a másolást kártyáról a szalagra. Kimenet hasonló. (+2 gép)
- Az I/O műveletek felgyorsulnak, a processzor kihasználtsága nő.
- A három számítógép párhuzamosan működik.
- Periféria-független programok, szabványos felületek.

Off-line I/O műveletek



a) Kártya a 1401-be
b) Kártya másolása szalagra
c) Szalag a 7094-hez
d) 7094 elvégzi a számítást
e) Szalag a 1401-be
f) 1401 kinyomtatja az eredményt

Programok

Tudományos és mérnöki számítások

- Parciális differenciál egyenletek
- (Könyvelés)

Merevlemez megjelenése

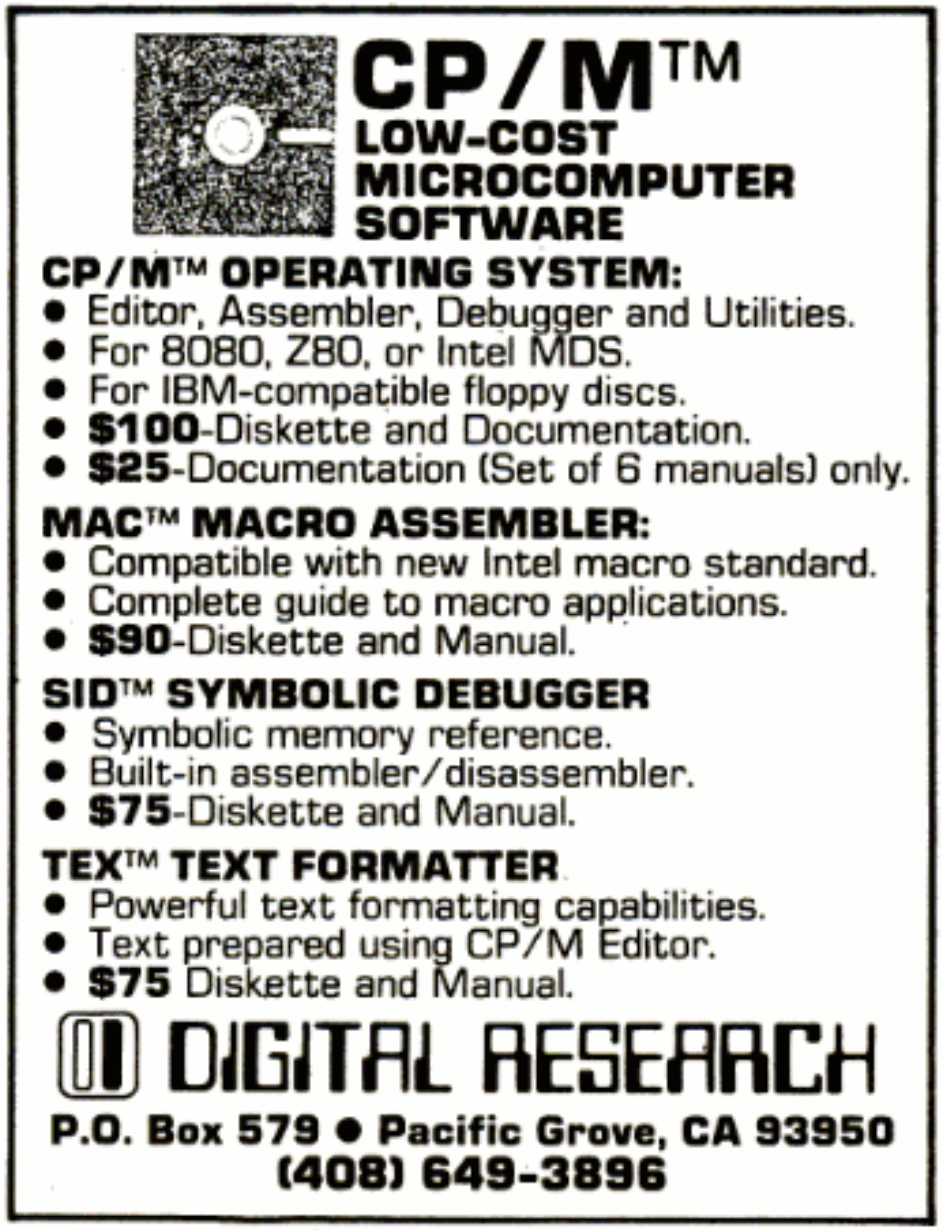
Egyszerű monitor: ős operációs rendszer

- A gép vezérlését egy állandóan a memóriában lévő program (a monitor) végzi, az operátor a perifériákat kezeli.
- A munkához rendelt vezérlő információkat a monitor értelmezi, végrehajtja, akár sorrendet is módosíthat a véletlen elérésnek köszönhetően
- Egy tevékenység befejezése után újra a monitor kapja meg a vezérlést, amely beolvassa a következő munkát.

CP/M

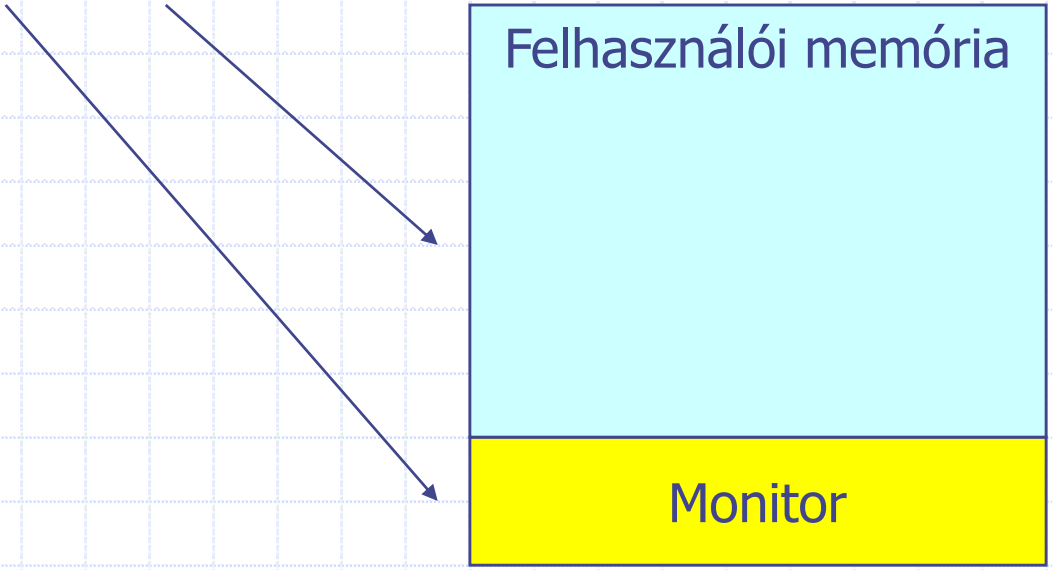
Control Program / Monitor

Disk Operating System

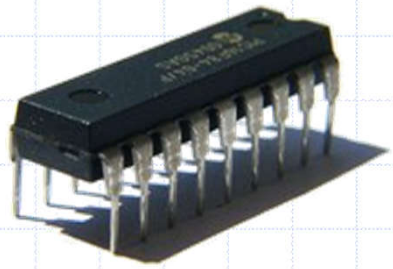


Kötegelt rendszer memória-kiosztása

Memória partíciók



2.3. Harmadik generáció



- Operációs rendszer megjelenik (1965-1980)
  - operátorok szerepe csökken
- Hardver
  - még kisebb, még olcsóbb
- A CPU sebessége jelentősen megnőtt
  - érdemes kihasználni az időt, amíg egy munka a lemezről olvasás eredményére vár
  - ezalatt az idő alatt egy másik program működhet

Definíció:

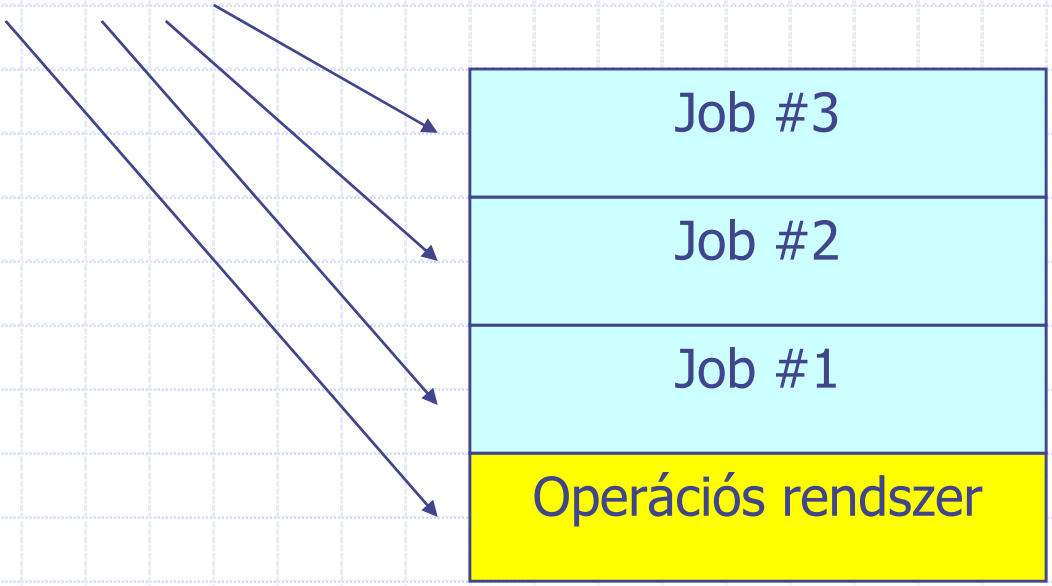
Multiprogramozás → az OS egyszerre több programot hajt végre

Programok

- Tudományos és mérnöki számítások
- Kereskedelmi alkalmazások

Multiprogramozott rendszer memória-kiosztása

Memória partíciók



A multiprogramozás lépései

- A rendszer nyilvántartja és tárolja a futtatandó munkákat.
- A kiválasztott munka addig fut, amíg várakozni nem kényszerül.
- Az OS feljegyzi a várakozás okát, majd kiválaszt egy másik futni képes munkát és azt elindítja.
- Ha a félbehagyott munka várakozási feltételei teljesülnek, akkor azt alkalmasint elindítja.

A multiprogramozás problémái

- Az átkapcsoláshoz több program van a tárban: *tárgazdálkodás*.
- Egy időben több futásra kész program: *CPU ütemezés.*
- A gépi erőforrások felhasználásának koordinációja: *allokáció, holtpont kezelés.*
- A programok ne zavarják egymást és az OS-t: *védelmi mechanizmusok*.

Perifériás műveletek gyorsítása

- A munkák között automatikus az átkapcsolás GYORS
- A perifériák sebessége a CPU-hoz képest LASSÚ
- Megoldások:
  - Off-line I/O műveletek (lásd korábban)
  - Pufferelés
  - Spooling (Simultaneous Peripheral Operations On-line)

Pufferelés

- CPU és a periféria vezérlő egyidejűleg működnek. A beolvasás pufferbe (központi memória) megy. Egy adott job adatfeldolgozási és I/O művelete átlapoltan történik.
- Pufferek mérete növekszik (blokkos átvitel).
- Csak akkor jelent hosszú távú sebesség növekedést, ha az I/O és a CPU műveletek kb. azonos időt vesznek igénybe (ami általában nem igaz).

Spooling

Spooling (Simultaneous Peripheral Operations On-line):

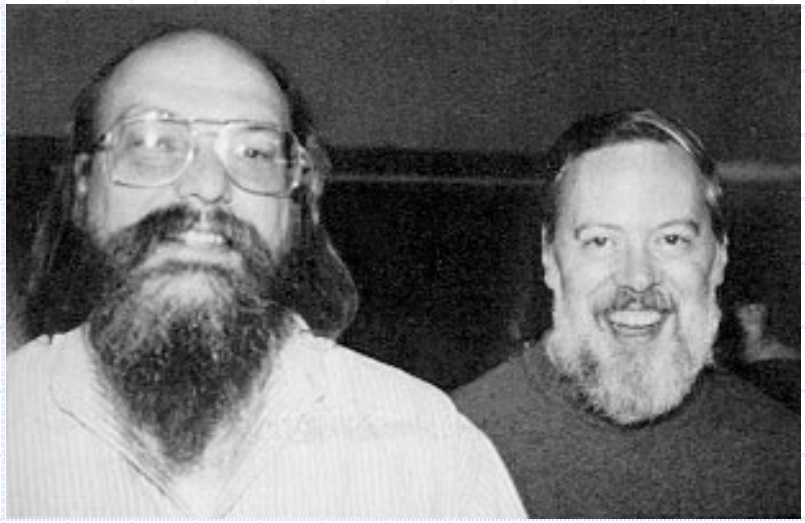
- A lassú perifériák adatait a feldolgozásig a kisméretű puffer helyett mágneslemezen tárolják.
- A partícióba a mágneslemezről kerül be.
- Különböző munkák perifériás és feldolgozási műveletei átlapolódhatnak.
- A lemez véletlen hozzáférése lehetővé teszi a munkák sorrendjének megválasztását.

Timesharing

- A harmadik generációs rendszerek is alapvetően még batch rendszerek voltak
- Egy job átfutása sokáig tartott
- Gyorsabb reakcióidőt szerettek volna a programozók
  - Azt az érzetet, hogy csak ők használják a gépet
- Terminál „minden” programozónak
- MULTICS (MULTiplexed Information and Computing Service) OS

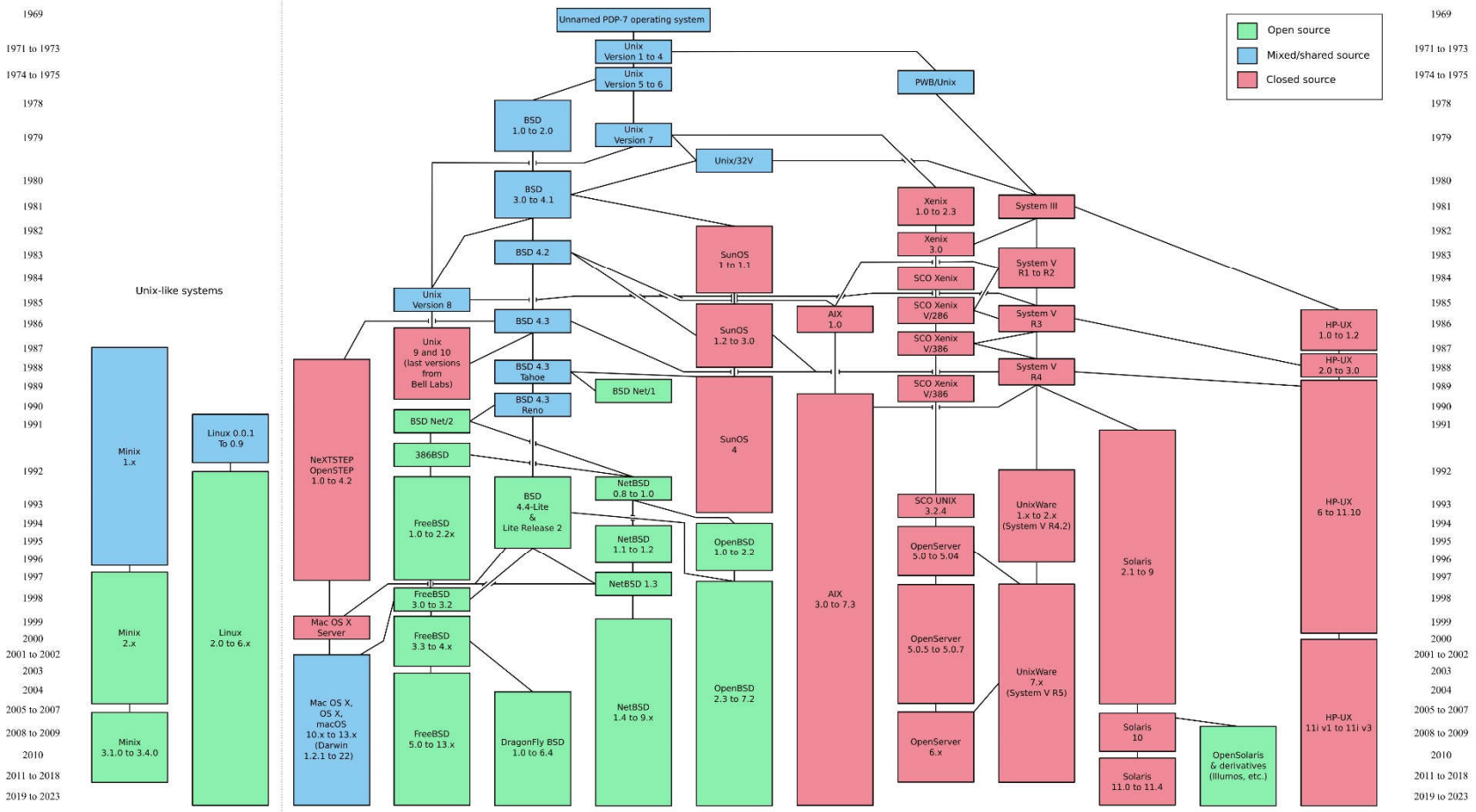
Unix

- Ken Thompshon
  Dennis Ritchie



- Uniplexed Information and Computing Service ☺
- Multics hibáit és bonyolultságát kiküszöböli
- C-ben írták → portolható

Unix


Unnamed PDP-7 operating system
Unix Version 1 to 4
Unix Version 5 to 6
Unix Version 7
PWB/Unix
Unix/32V
BSD 1.0 to 2.0
BSD 3.0 to 4.1
BSD 4.2
BSD 4.3
BSD 4.3 Tahoe
BSD 4.3 Reno
Unix Version 8
Unix 9 and 10 (last versions from Bell Labs)
BSD Net/1
BSD Net/2
386BSD
BSD 4.4-Lite & Lite Release 2
NeXTSTEP OpenSTEP 1.0 to 4.2
Mac OS X Server
Mac OS X, OS X, macOS 10.x to 13.x (Darwin 1.2.1 to 22)
FreeBSD 1.0 to 2.2x
FreeBSD 3.0 to 3.2
FreeBSD 3.3 to 4.x
FreeBSD 5.0 to 13.x
DragonFly BSD 1.0 to 6.4
NetBSD 0.8 to 1.0
NetBSD 1.1 to 1.2
NetBSD 1.3
NetBSD 1.4 to 9.x
SunOS 1 to 1.1
SunOS 1.2 to 3.0
SunOS 4
OpenBSD 1.0 to 2.2
OpenBSD 2.3 to 7.2
AIX 1.0
AIX 3.0 to 7.3
Xenix 1.0 to 2.3
Xenix 3.0
SCO Xenix
SCO Xenix V/286
SCO Xenix V/386
SCO UNIX 3.2.4
OpenServer 5.0 to 5.04
OpenServer 5.0.5 to 5.0.7
OpenServer 6.x
System III
System V R1 to R2
System V R3
System V R4
UnixWare 1.x to 2.x (System V R4.2)
UnixWare 7.x (System V R5)
Solaris 2.1 to 9
Solaris 10
Solaris 11.0 to 11.4
OpenSolaris & derivatives (Illumos, etc.)
HP-UX 1.0 to 1.2
HP-UX 2.0 to 3.0
HP-UX 6 to 11.10
HP-UX 11i v1 to 11i v3
Unix-like systems
Minix 1.x
Minix 2.x
Minix 3.1.0 to 3.4.0
Linux 0.0.1 To 0.9
Linux 2.0 to 6.x
Open source
Mixed/shared source
Closed source
1969
1971 to 1973
1974 to 1975
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001 to 2002
2003
2004
2005 to 2007
2008 to 2009
2010
2011 to 2018
2019 to 2023

2.4 Negyedik generáció

- 1980- (*főleg személyi számítógépek*)
- Hardver: kicsi, olcsó, nem csak cégeknek megfizethető
- Korszerű kötegelt rendszerek
- Időosztásos rendszerek
- Valós idejű rendszerek
- Elosztott operációs rendszerek
- Beágyazott operációs rendszerek

Korszerű kötegelt rendszerek

Csak nagy számítógépeken !!

- A gyors véletlen-hozzáférésű mágneslemezek miatt megszűnt a kötegelt rendszerek szükségessége.
- A jelenlegi rendszerekben a programok futásába nem lehet interaktívan beavatkozni, csak előre összeállított munkák futnak.
- Hátrányok:
  - Átfutási idő hosszú. (Egy munkának a rendszerbe adása és az eredmény közti idő.)
  - A programok csak statikusan, az eredmények alapján fejleszthetők, használhatók. Az átfutási idő megnehezíti a programokkal történő kísérletezést.

Időosztásos rendszerek

- A időosztásos rendszer (timesharing, multitask systems) *közvetlen interaktív kommunikációt* biztosít a felhasználó és a programja, ill. az OS között.
- Adatok tárolása közvetlenül (on-line) az állományrendszerben.
- A felhasználó interakciója nagyon lassú, közben az OS más tevékenységet tud végrehajtani.
- Gyors reakció a parancsokra, válaszidő (response time) kicsi. Sűrűn kell a programok között kapcsolgatni.
- Felhasználók függetlenül használják a gépet, mintha mindenki egy saját gépen dolgozna.

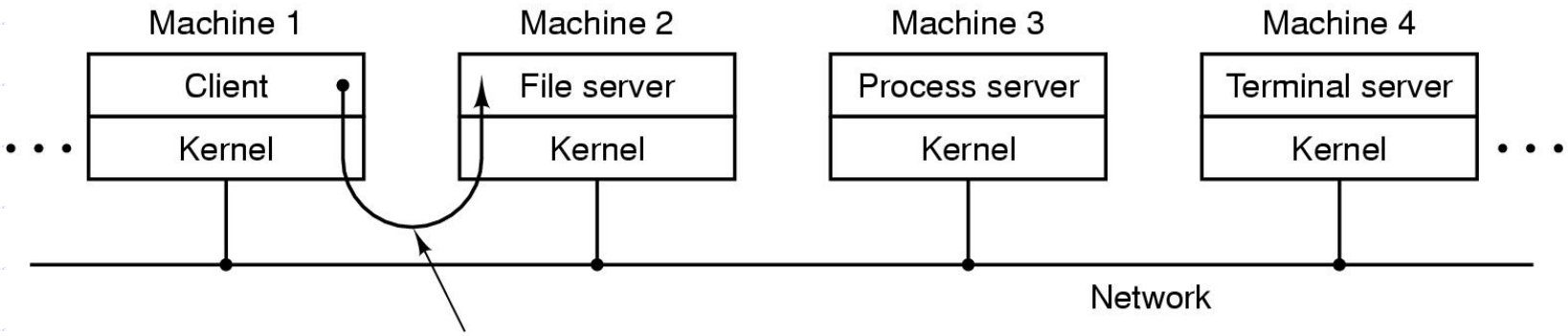
Elosztott operációs rendszerek

- A számításokat több központi egység között osztják meg egyenletesen (multiprocessing)
- A felhasználó ezt nem veszi észre (transzparens)
- Szervezési elvek:
  - Azonosak-e a processzorok
    - homogén ↔ inhomogén
  - Van-e kitüntetett processzor:
    - szimmetrikus ↔ aszimmetrikus
  - Csatolás
    - szorosan csatolt (tár egy részét közösen használják)
    - nem szorosan csatolt (a CPU-kat kommunikációs csatorna köti össze) [elosztott rendszerek általában ilyenek]

Elosztott operációs rendszerek kommunikációja

- Hálózati infrastruktúra kell (LAN vagy WAN)
- Architektúra: kliens-szerver, vagy peer-to-peer

Példa: egy kliens-szerver architektúra:



Üzenet egy klienstől egy szerver számára

Elosztott operációs rendszerek előnyei

- Erőforrások megosztása
  - A felhasználó a másik gép erőforrásait (spec. hardver, perifériák, állományok adatbázisok) használhatja
- Terhelés megosztás
  - párhuzamosan végrehajtható részeket szét lehet osztani
  - megnövelt teljesítmény
- Megbízhatóság
  - Egy elem kiesése esetén a többi elem átveheti a szerepét.
    - Homogén esetén könnyebb, inhomogén esetében nehezebb
    - *„graceful degradation”*
- Kommunikáció
  - Programok, felhasználók közötti információ csere.

Valósidejű rendszerek

- Kemény valós idejű rendszerek (hard real-time)
  - Szigorúan definiált és betartott válaszidők
  - Kritikus rendszerek (pl. atomreaktor, járművek)
  - Adattárolás: RAM, ROM, másodlagos tárolást (diszk) nem támogatják
  - Általános célú operációs rendszerek nem támogatják
- Puha valós idejű rendszerek (soft real-time)
  - A válaszidők betartására törekednek, csúszás megengedett
  - Nem kritikus folyamatirányítási feladatok
  - Multimédia rendszerek, virtuális valóság

Beágyazott operációs rendszerek

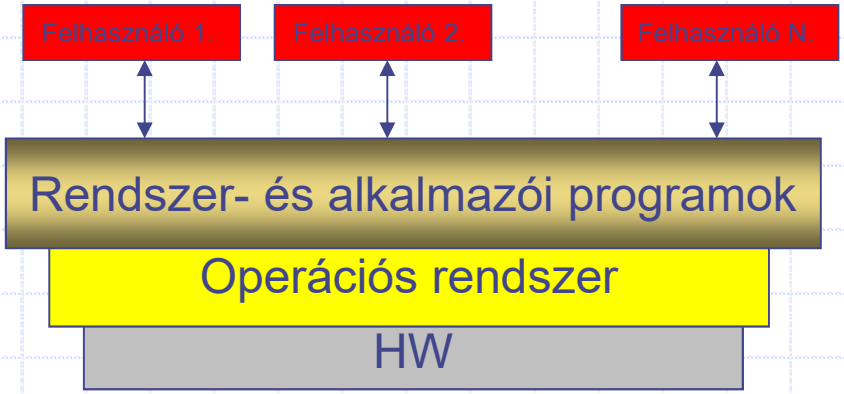
- Háztartási berendezések, mobiltelefonok, járművek, játékok
- Nem általános célú, nem független az alkalmazástól (összefordul)
- Korlátos erőforrások, korlátos szolgáltatások. OS-nek csak a szükséges része fordul be.

3. Az operációs rendszerek szerkezete

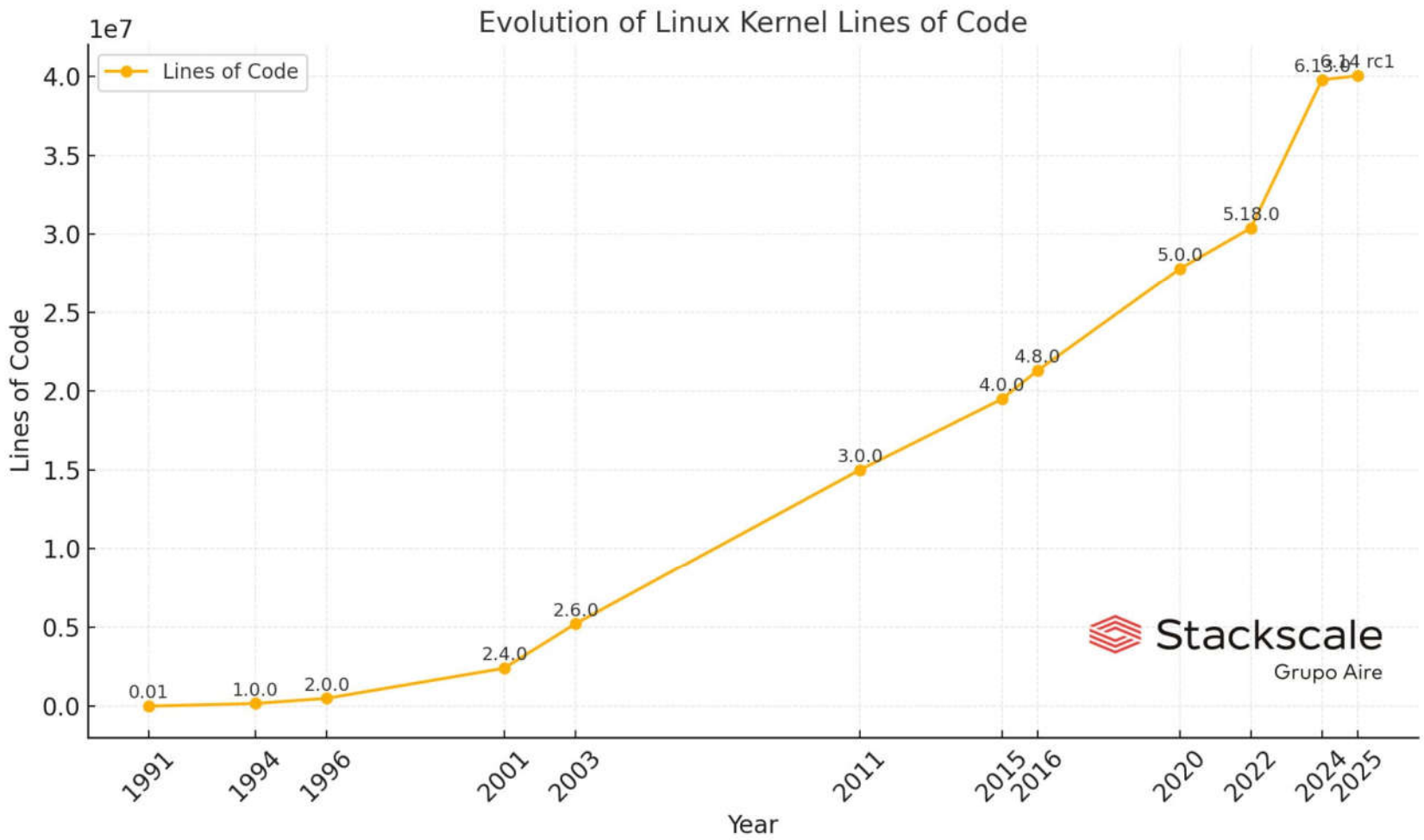
- OS:
  - nagyméretű
  - komplex program,
  - belső szerkezetük részekből áll.

- A felosztásnál *rétegeket* különböztetünk meg:
  - egymásra épülnek
  - az alattuk elhelyezkedő réteggel jól definiált felülettel érintkeznek
  - egyre bővülő funkcionalitás.

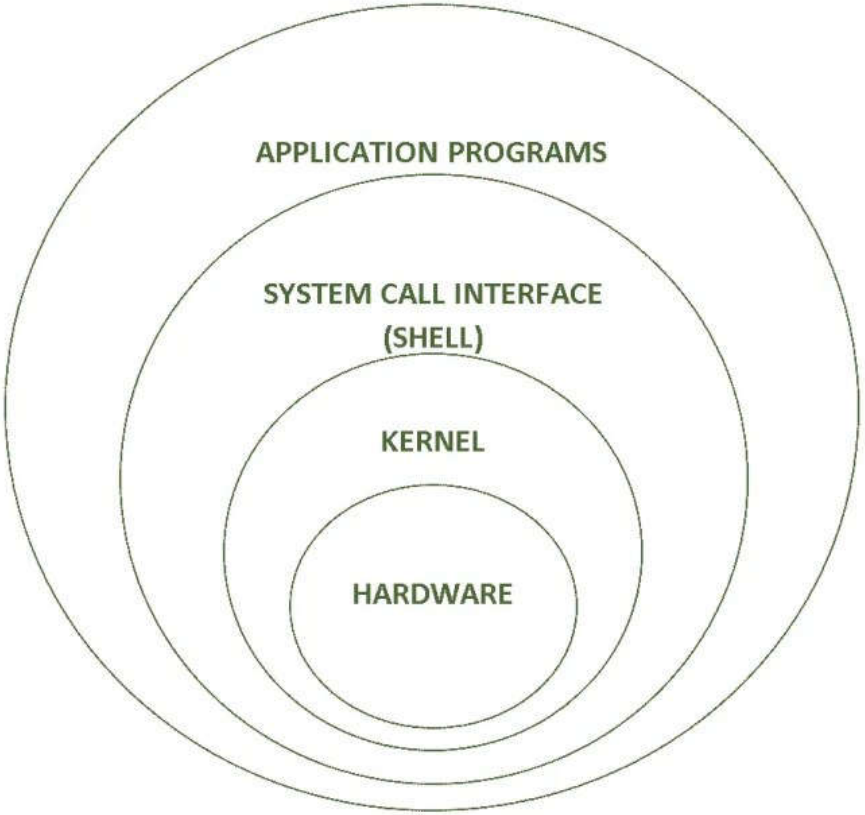
- Legalább 3 réteg:
  - hardver
  - operációs rendszer
  - felhasználói programok



How large is large?



Unix alapvető rétegszerkezete



UNIX Architecture

Created by Notes Jam

Rétegszerkezet rétegei

1. felhasználói programok
2. periféria meghajtók
3. virtuális tár kezelése
4. I/O csatornák
5. CPU ütemezése
6. hardver

Tendencia a *kevés réteg*.

Rétegszerkezet

- Tiszta rétegszerkezetű rendszerek: csak az alatta és felette levő réteggel kommunikál, a rétegek feladatainak egymásra épülését megnehezíti.
- Modulok, a rétegen belüli strukturálás: egy-egy jól meghatározott feladatcsoportra egy modul.
- A modulok egymás között kommunikálnak, egységes mechanizmus.

CP/M rétegszerkezet

- Basic Input/Output System (BIOS),
- Basic Disk Operating System (BDOS),
- Console Command Processor (CCP)

(BIOS, DOS, COMMAND.COM)

Linux rétegszerkezet

User mode	User applications	*bash, LibreOffice, GIMP, Blender, 0 A.D., Mozilla Firefox, ...*				
	System components	**init daemon:** *OpenRC, runit, systemd...*	**System daemons:** *polkitd, smbd, sshd, udevd...*	**Windowing system:** *X11, Wayland, SurfaceFlinger (Android)*	**Graphics:** *Mesa, AMD Catalyst, ...*	**Other libraries:** *GTK, Qt, EFL, SDL, SFML, FLTK, GNUstep, ...*
	C standard library	`fopen`, `execv`, `malloc`, `memcpy`, `localtime`, `pthread_create`... (up to 2000 subroutines) *glibc* aims to be fast, *musl* aims to be lightweight, *uClibc* targets embedded systems, *bionic* was written for Android, etc. All aim to be POSIX/SUS-compatible.				
Kernel mode	**Linux kernel**	`stat`, `splice`, `dup`, `read`, `open`, `ioctl`, `write`, `mmap`, `close`, `exit`, etc. (about 380 system calls) The Linux kernel System Call Interface (SCI), aims to be POSIX/SUS-compatible[3]				
		Process scheduling subsystem	IPC subsystem	Memory management subsystem	Virtual files subsystem	Networking subsystem
		Other components: ALSA, DRI, evdev, klibc, LVM, device mapper, Linux Network Scheduler, Netfilter Linux Security Modules: *SELinux*, *TOMOYO*, *AppArmor*, *Smack*				
Hardware (CPU, main memory, data storage devices, etc.)						

4. Az operációs rendszerek feladatai

- Rendszerhívások
- Fontosabb rendszermodulok
- Szolgáltatások
- Rendszerprogramok

4.1 Rendszerhívások

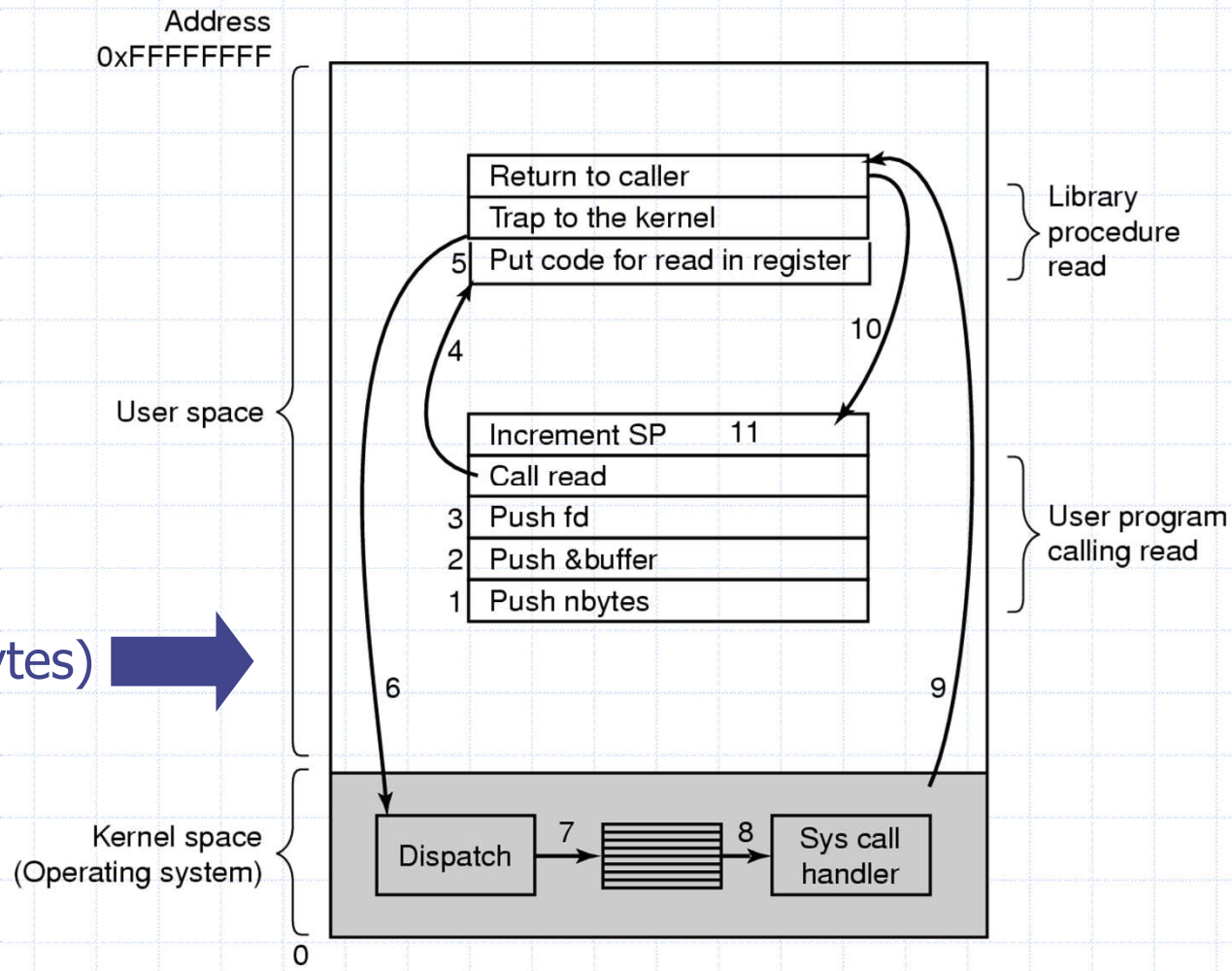
- Program és OS közötti felület szigorúan rögzített.
- A programok csak ezt látják.
- A rendszer szolgáltatásainak igénybevétele: *rendszerhívások*.
- Rendszerhívás speciális gépi utasítás segítségével történik (trap). A vezérlést az OS programjának adja.
  - assembly-ből közvetlenül kiadhatók (pl. DOS int010 megszakítás)
  - magasszintű nyelveknél könyvtári eljárások

A rendszerhívás lépései

- Paraméter átadás (1,2,3)
  - Az OS által megadott helyre kell elhelyezni.
- Hívás, hardver működési mód váltás (4,5,6)
  - felhasználói mód (korlátozott, bizonyos utasítások nem) → rendszer mód (módváltás).
  - A rendszerhívó gépi utasítás végzi el.
- Paraméterek kiolvasása, elágazás a hívott szolgáltatáshoz (a rendszerhívás célját megvalósító kód végrehajtása) (7)
- Visszaadott paraméterek másolása (8)
- Visszatérés a hívóhoz (módváltás) (9)
- Hívó tovább fut, eredmények felhasználása (10,11)

A rendszerhívás lépései

read (fd, buffer, nbytes)



Példák rendszerhívásokra

Miscellaneous

Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

4.2 Fontosabb rendszermodulok

- Folyamatok kezelése
- Központi tár kezelése
- Perifériák kezelése
- Állományok kezelése
- Védelmi mechanizmusok
- Hálózatok kezelése
- Kezelői felület

Folyamatok kezelése 1.

Problémák

- Folyamatokhoz erőforrások biztosítása
  - Folyamat indulásakor allokálódnak, vagy amikor szükség van rá.
- Holtpont a folyamatok között (kölcsönös várakozás).
- CPU mint erőforrás a folyamatok közötti megosztása (CPU ütemezés).

Folyamatok kezelése 2.

Folyamatokat kezelő rendszerhívások:

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

Központi tár kezelése

- Folyamathoz annak indulásánál (esetleg később is) memóriát kell rendelni.
- Virtuális tárkezelés
  - virtuális cím - közvetlen cím hozzárendelés
- Központi tárban több folyamat utasításai, adatai vannak. Ezért biztosítani kell, hogy
  - a futáshoz szükséges részek az operatív tárban,
  - a nem szükségesek pedig a háttértáron legyenek.

Perifériák kezelése

- Perifériák elrejtése a programok elől.
- Átvitel pufferelése
  - CPU jobb kihasználása érdekében.
- Periféria (HD) jobb kihasználása érdekében több egyidejű igény ütemezése.

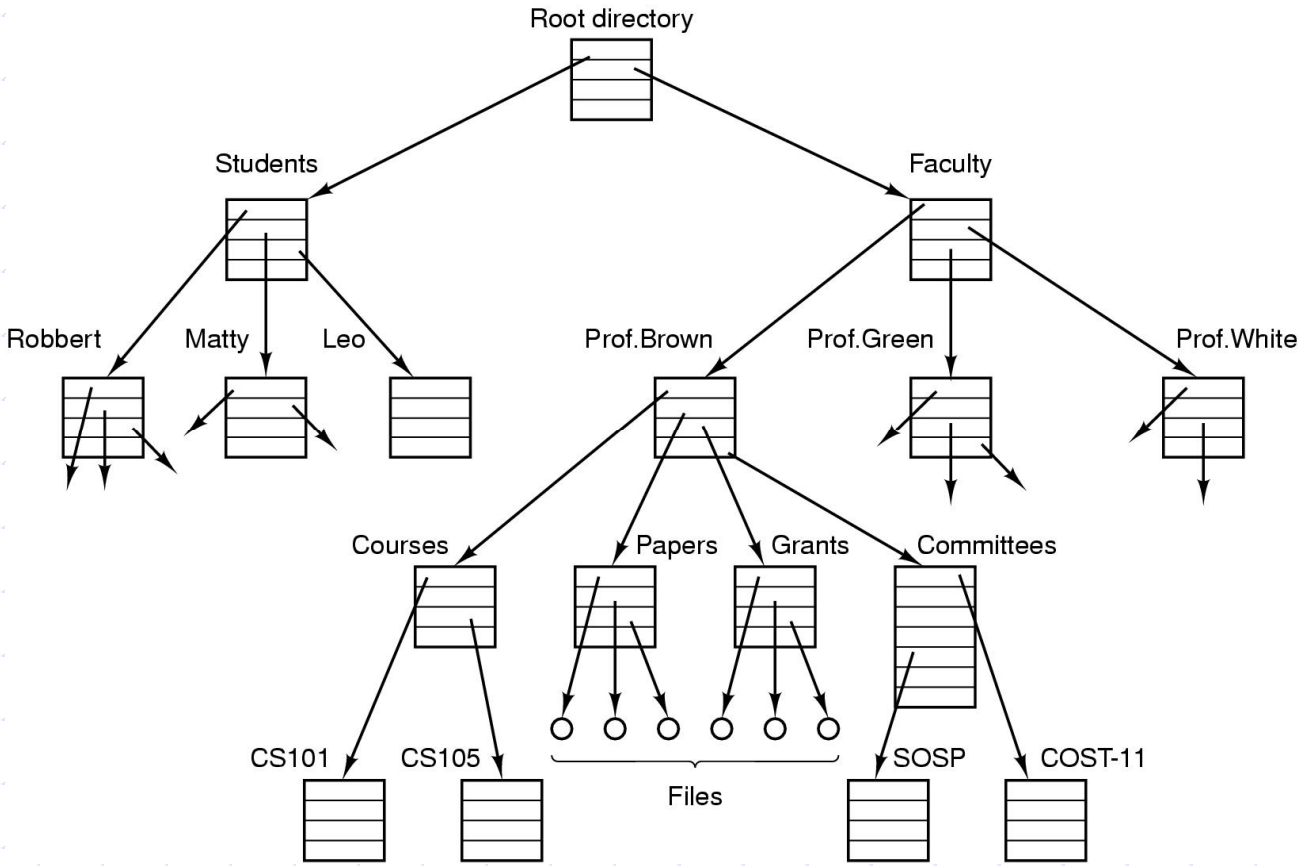
Állományok kezelése

Fájl (file):

- egymáshoz tartozó információk gyűjteménye
- elrejti a tárolás részleteit.

- Tárolja a felhasználói programokat és adatokat.
- állományok csoportosítása könyvtárakban (hierarchia)
- állományok létrehozása, megszüntetése, olvasása, írása, stb., valamint a szabad hely nyilvántartása.
- Megbízható mentés (backup).

Állományok felépítése



Állománykezelő rendszerhívások

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Védelmi mechanizmusok

- OS-t és erőforrásait védeni kell a hibás vagy illetéktelen felhasználásoktól.
- Lehet
  - hardver, vagy
  - OS-ben megvalósított

  védelmi mechanizmus

Hálózatok kezelése

- Távoli felhasználókkal, gépekkel való *kommunikáció,*
- távoli erőforrások elérése.

Kezelői felület

Definíció:

A kezelői felület célja az OS és a felhasználó közötti kommunikáció biztosítása.

- Karakteres kommunikáció (parancsértelmező, parancsnyelv) – CLI
- Grafikus, speciális beviteli eszközökkel megvalósított kommunikáció – GUI

4.3 Az operációs rendszerek szolgáltatásai

- Folyamatok vezérlése
- Állományok kezelése
- Perifériás eszközök kezelése
- Rendszerinformációk kezelése
- Kommunikáció

Folyamatok vezérlése

- program betöltése, végrehajtása
- folyamatok létrehozása, megszüntetése; attribútumainak beállítása;
- központi tár igénylése, felszabadítása;
- folyamatok közötti kommunikáció, folyamatok szinkronizálása (egymáshoz, órához, külső eseményhez);
- nyomkövetés hibakereséshez

Állományok kezelése

- állomány létrehozása, törlése, attribútumok beállítása;
- könyvtárak létrehozása, módosítása;
- állományok megnyitása, lezárása, szekvenciális és véletlen átvitel .

Perifériás eszközök kezelése

- Perifériák
  - igénylése;
  - lefoglalása;
  - felszabadítása;
- Adatátvitel a program és a periféria között

Rendszerinformációk kezelése

- A rendszerkomponensek
  - folyamatok
  - állományok
  - perifériák

  állapotának lekérdezése, módosítása;
- rendszer statisztikák, naplózás

Kommunikáció

- kommunikációs csatornák
  - létrehozása
  - megszüntetése;
- üzenetek
  - küldése
  - fogadása;
- műveletek távoli erőforrásokon.

4.4 Rendszerprogramok

Definíció:

Az OS-hez adott, annak általános célú felhasználását támogató programok.

- parancsértelmező
- állományrendszer kezelő
- szövegszerkesztő
- fordító program, linker, loader, könyvtár kezelő
- kommunikációs programok (pl. levelezés)